

RaoRocketSim

A Differentiable Rao Thrust-Optimised-Parabola Nozzle Design Tool
Reference Manual: Functionality, Mathematics, and Physics

Ibrahim Shahid

June 2026 — J4 gate closed ($\text{max_scaled} = 1.16 \times 10^{-3} \leq 2 \times 10^{-3}$)

Abstract

RaoRocketSim computes thrust-optimum (Rao/TOP) axisymmetric rocket nozzle contours by solving Rao’s variational problem directly as a discretised boundary-value problem (BVP), with the method of characteristics (MOC) supplying the transonic kernel and the wall construction. The solver is implemented twice over a single physics definition: a NumPy/SciPy reference path and a JAX path with *exact* automatic-differentiation Jacobians solved by Levenberg–Marquardt (Optimistix), held to 10^{-15} -level residual parity. On the reference design point ($\varepsilon = 10$, 80% bell, $\gamma = 1.4$) the solver converges to max scaled residual 1.16×10^{-3} with mass and length constraints closed to $\mathcal{O}(10^{-9})$, and emits a closed, monotone wall contour ending exactly on the commanded exit via the NASA BDE-region construction. All geometry is **preliminary design output**: not CFD-, thermal-, structurally-, or hot-fire-validated.

Contents

1	Repository functionality overview	1
1.1	What the tool does	1
1.2	Module map	1
2	Governing gas dynamics	2
2.1	Isentropic relations	2
2.2	Prandtl–Meyer function and Mach angle	2
2.3	Critical Mach number	2
2.4	Area–Mach relation	3
3	Method of characteristics (axisymmetric)	3
3.1	Transonic start line (Kliegel–Levine)	3
4	Rao’s variational problem	3
5	NASA topology constructions	4
6	The BVP formulation	4
6.1	Unknowns	4
6.2	Residual blocks (characteristic formulation)	5
7	The differentiable (JAX) backend	5

8	Convergence campaign (the J4 record)	6
9	Parametric contour generation	6
10	Verification and reliability	7
11	Remaining roadmap (formalised final steps)	7

1 Repository functionality overview

1.1 What the tool does

Given a design point $(R_t, \varepsilon = A_e/A_t, \text{length}\%, \gamma, p_a/p_0)$, RaoRocketSim:

1. builds the transonic start line TT' (Kliegel–Levine) and marches the supersonic *kernel* with axisymmetric MOC;
2. seeds and solves Rao’s variational problem for the optimum control surface DE as a coupled residual BVP;
3. constructs the bell wall from the converged solution via the NASA BDE-region march;
4. exports the contour (CSV / binary STL / plots) and reports a residual-backed reliability grade.

1.2 Module map

Module	Responsibility
<code>raosim/gas_dynamics.py</code>	Isentropic ratios, Prandtl–Meyer function and inverse, Mach angle, area–Mach relation, critical Mach M^* , ideal thrust coefficient.
<code>raosim/moc.py</code>	Axisymmetric MOC unit processes (interior, axis, wall points), characteristic-net marching, compatibility auditing.
<code>raosim/transonic_kernel.py</code>	Mathematically corrected Kliegel–Levine third-order transonic evaluator.
<code>raosim/nasa_moc.py</code>	Port of the NASA/JHU <code>MOC_GridCalc_BDE.cpp</code> workflow: <code>KLThroat</code> (visible-source semantics), <code>CalcInitialThroatLine</code> , <code>RRC</code> kernel march (<code>build_kernel</code>), <code>CalcMdotBD</code> , <code>FindPointE</code> , <code>CalcLRCDE</code> (free-exit and <code>FIXEDEND</code> branches), <code>SetThetaB</code> , BDE-region wall (<code>calc_bde_region</code>). Oracle-validated against the checked-in <code>outputs.M3.5Perf</code> reference grids.
<code>raosim/rao_variational.py</code>	The Rao BVP: configuration, unknown packing, residual blocks (legacy and <i>characteristic</i> formulations), seeding, the solve shell, wall construction dispatch, reliability ladder.
<code>raosim/jax/</code>	Differentiable backend: <code>primitives</code> (J1-parity gas dynamics), <code>residuals</code> (vectorised leaves), <code>assembly</code> (full residual, 10^{-15} parity), <code>bvp/ api</code> (Optimistix LM, sigmoid-bounded, constraint-weight ladder).

<code>raosim/nozzle_geometry.py</code>	Chart-calibrated Bézier Rao/TOP approximation (published θ_N, θ_E tables); conical nozzles.
<code>raosim/export.py</code> , <code>plotting.py</code> , <code>trajectory.py</code> , ...	CSV/STL export, plots, altitude performance, trade studies, CLI.
<code>scripts/generate_contour.py</code>	Parametric end-to-end generator (§9).
<code>scripts/j4_report.py</code>	One-shot solver status report.

2 Governing gas dynamics

Calorically perfect, inviscid, irrotational, axisymmetric flow with stagnation-normalised state (p/p_0 , T/T_0 , ρ/ρ_0).

2.1 Isentropic relations

$$\frac{T_0}{T} = 1 + \frac{\gamma - 1}{2} M^2, \quad \frac{p}{p_0} = \left(\frac{T}{T_0}\right)^{\frac{\gamma}{\gamma-1}}, \quad \frac{\rho}{\rho_0} = \left(\frac{T}{T_0}\right)^{\frac{1}{\gamma-1}}. \quad (1)$$

2.2 Prandtl–Meyer function and Mach angle

$$\nu(M) = \sqrt{\frac{\gamma+1}{\gamma-1}} \arctan \sqrt{\frac{\gamma-1}{\gamma+1} (M^2 - 1)} - \arctan \sqrt{M^2 - 1}, \quad \mu = \arcsin(1/M). \quad (2)$$

2.3 Critical Mach number

$$M^{*2} = \frac{(\gamma + 1)M^2}{2 + (\gamma - 1)M^2}, \quad (3)$$

the velocity normalised by the throat speed of sound; it appears in Rao’s algebraic stationarity (§4).

2.4 Area–Mach relation

$$\frac{A}{A^*} = \frac{1}{M} \left[\frac{2}{\gamma + 1} \left(1 + \frac{\gamma - 1}{2} M^2 \right) \right]^{\frac{\gamma+1}{2(\gamma-1)}}. \quad (4)$$

3 Method of characteristics (axisymmetric)

Characteristic directions $dr/dx = \tan(\theta \pm \mu)$ ($C^\pm$). Compatibility along them (Anderson, *Modern Compressible Flow*, Ch. 11):

$$d(\theta + \nu) = Q^+ ds \quad \text{on } C^+, \quad d(\theta - \nu) = Q^- ds \quad \text{on } C^-, \quad (5)$$

$$Q^\pm = \pm \frac{\sin \theta \sin \mu \cos \mu}{r \cos(\theta \pm \mu)}. \quad (6)$$

The axisymmetric source terms $Q^\pm$ make $\theta \pm \nu$ *local* quantities, not global invariants. Unit processes (interior, axis, wall) solve the two-equation system per node; the NASA port uses the $d\theta$ -form derivative system of `MOC_GridCalc_BDE.cpp`.

3.1 Transonic start line (Kliegel–Levine)

The kernel cannot start at $M = 1$ exactly; the TT' line comes from the third-order Kliegel–Levine expansion in inverse powers of $(R_u/R_t + 1)$, where R_u is the *upstream* wall radius of curvature (Hall 1962; Kliegel & Levine 1969). Two port subtleties are load-bearing and oracle-pinned by `tests/test_nasa_kernel_march_parity.py`:

- **C++ integer division:** the visible source computes $z(y^2 - 5/8)$ with $5/8 = 0$; the binary that generated all reference fixtures ran with the term dropped. The “visible-source” evaluator reproduces this exactly (TT' matches TT'.out to 5×10^{-7}); the theory-correct $y^2 - 5/8$ lives in `transonic_kernel.py`.
- **Upstream radius:** `CalcInitialThroatLine(rUp, ...)` feeds the *upstream* radius into `KLThroat`. Rao convention: $R_u = 1.5R_t$ upstream, $R_d = 0.382R_t$ downstream (`throat_upstream_radius_factor`).

With both honoured, `build_kernel` reproduces NASA’s M3.5Perf kernel: 58 RRC rows, BD matching `LastKernel.out` node for node.

4 Rao’s variational problem

Rao (1958) maximises thrust over exit contours of fixed length and area ratio. The optimum is characterised on a *control surface* DE — a left-running (C^+) characteristic from a kernel point D to the exit lip E — by integrals of the flow:

$$F = \int_{DE} 2\pi r \left[\rho V^2 \cos \theta \frac{\sin(\phi - \theta)}{\sin \phi} + (p - p_a) \right] dr \quad (\text{thrust}), \quad (7)$$

$$\dot{m} = \int_{DE} 2\pi r \rho V \frac{\sin(\phi - \theta)}{\sin \phi} dr \quad (\text{mass flow}), \quad (8)$$

$$L = z_C + \int_{DE} \cot \phi dr \quad (\text{exit station; } z_C = x_D). \quad (9)$$

The Euler–Lagrange conditions collapse to the *algebraic Rao stationarity* (Rao–Beck–Booth, AIAA 99-2584, Eq. 3):

$$\boxed{\frac{M^* \cos(\theta - \alpha)}{\cos \alpha} = C \quad \text{constant along DE,} \quad \alpha = \arcsin(1/M).} \quad (10)$$

Mass closure ties DE to the kernel: the flux crossing DE equals the flux crossing the kernel characteristic segment B→D,

$$\dot{m}_{DE} = \dot{m}_{BD}, \quad \dot{m}_\Gamma = \int_\Gamma 2\pi r \rho V |\sin(\beta - \theta)| ds, \quad (11)$$

with β the local polyline tangent. The smooth optimum exists only inside Guderley’s validity region; the solver checks the inequality (`rao_valid_region`) and reports `requires_discontinuous_exit_flow_model` otherwise.

Length bookkeeping (a load-bearing correction). The constraint is the *exit station* $x_E = z_C + \int \cot \phi dr$. An earlier implementation compared $\sum dx = x_E - x_D$ against L while a separate residual pinned $x_E = L$ — contradictory unless $x_D \rightarrow 0$, which silently drove the solved D to the throat plane. Restoring z_C moved the reference solve from max scaled 6.2×10^{-2} to 3.0×10^{-3} in one step.

5 NASA topology constructions

Ports of `MOC_GridCalc_BDE.cpp` (Rice, JHU/APL 2003):

`CalcMdotBD` mass flux from B to a candidate D along the kernel's final RRC (annular trapezoids of ρu).

`FindPointE` integrates the C^+ derivative system (RKF45 in r) from D, accumulating annular mass until $\dot{m}_{DE} = \dot{m}_{BD}$; the terminus is E.

`CalcLRCDE` places D on BD. Two inner conditions: *free-exit* (`rao_free`) zeroes $\theta_E - \frac{1}{2} \arcsin[2(p_E - p_a)/(\rho_E V_E^2 \tan \mu_E)]$ (Rao Eq. 14); *FIXEDEND* (`fixed_end`) walks D until r_E crosses the commanded exit radius, then secants. The fixed- (L, ε) design point uses *FIXEDEND*.

`SetThetaB` outer secant on the initial expansion angle θ_B driving the remaining exit parameter (length), with SEC_FAIL-style bracketing. Note the monotonicity: *lower* $\theta_B \Rightarrow$ *longer* nozzle.

`calc.bde_region` back-computes the region bounded by BD, DE and the wall, then locates each wall point by mass conservation (`CalcWallContour`) — this is the wall used by `wall_method="bde"`.

6 The BVP formulation

6.1 Unknowns

$$u = [M_{1..n}, \theta_{1..n}, r_{1..n} \mid \lambda_2, \lambda_3, \log C, f_D], \quad (12)$$

where $f_D \in [0, 1]$ is D's arc-length fraction along the (static) kernel BD. x is *reconstructed*, not carried: DE is a C^+ characteristic by construction,

$$x_{i+1} = x_i + \frac{\Delta r}{\tan(\bar{\theta} + \bar{\mu})}, \quad x_1 = x_D(f_D), \quad (13)$$

making left-Mach geometry exact to machine precision.

6.2 Residual blocks (characteristic formulation)

block	content	scaling
<code>mass</code>	$(\dot{m}_{DE} - \dot{m}_{BD})/\dot{m}_{\text{ref}}$	relative
<code>length</code>	$(x_E - L)/L$ (exit station)	relative
<code>algebraic_stationarity</code>	$\log M^* + \log \cos(\theta - \alpha) $ $- \log \cos \alpha - \log C$	per node
<code>moc_cplus</code>	$\Delta(\theta + \nu) - Q^+ \Delta s$ per segment	$/1^\circ$
<code>ce_geometry</code>	D attachment (r pin; optional θ , M pins), exit pin $(x_E, r_E) = (L, R_e)$, monotonicity	mixed
<code>regularization</code>	$0.02 \Delta^2(\theta \pm \nu)$	$/1^\circ$
<code>penalties</code>	one-sided monotonicity guards	—

Deliberately *absent* relative to the legacy stack (each removal literature-grounded and empirically decisive): `moc_cminus` on DE (the C^- relation does not hold along a C^+ characteristic; Rao's closure carries only C^+ relations + stationarity + integrals) and the CE→wall C^+ pairing blocks (the C^+ through a DE point *is* DE; the wall belongs to the BDE region). The D attachment is position-only at the gate (`pin_d_theta=False`): pinning θ_1 to the *interpolated, approximate* kernel angle imports kernel discretisation error into the stationarity chain at full weight.

7 The differentiable (JAX) backend

The entire residual is re-expressed in `jax.numpy` (`raosim/jax/assembly.py`) and held to $\sim 10^{-15}$ parity with the NumPy assembly on real solver states (J2 gate). The solve is Levenberg–Marquardt (Optimistix) with *exact* `jacfwd` Jacobians — replacing SciPy’s finite-difference Jacobian, whose noise near the sonic and Mach-line singularities was the original convergence suspect. Box bounds are enforced by a smooth sigmoid reparametrisation $u = \ell + (h - \ell) \sigma(z)$, and the integral constraints (single residual elements vs. $\mathcal{O}(n)$ physics elements) are up-weighted along a continuation ladder (1, 10, 30, 100), each rung warm-starting the next; reported residuals are always unweighted.

```
cfg = RaoSolverConfig(
    Rt=0.020, epsilon=10.0, gamma=1.4, pa_over_p0=0.01, length_pct=80.0,
    n_control=24, n_kernel=24, max_nfev=4000, residual_tol=2e-3,
    evaluate_moc=True, wall_method="bde",
    solver_backend="jax", formulation="characteristic",
    pin_d_theta=False, thetaN_guess_deg=21.87,
    jax_constraint_weight_ladder=(1.0, 10.0, 30.0, 100.0))
sol = solve_rao_bvp(cfg) # sol.residuals.max_scaled -> 1.16e-3
```

8 Convergence campaign (the J4 record)

max scaled	change introduced	root cause closed
≈ 8	(baseline) SciPy FD Jacobian	—
2.8	exact-Jacobian LM (JAX)	FD noise removed
0.50	KLThroat int-division + R_u fixes	kernel march never ran
0.49	NASA FIXEDEND topology seed	degenerate D $\approx$ E seed
0.062	characteristic formulation + ladder	C^- -on- C^+ scaffolds
0.0030	exit-station length (z_C restored)	x_D double-count
0.00116	position-only D attachment	θ_D interpolation error

At the gate: mass -1.9×10^{-9} , length -5.8×10^{-10} , $f_D = 0.304$ (the classical interior band), $\theta_B = 21.87^\circ$ — independently recovering the published chart angle $\theta_N(\varepsilon=10, 80\%) \approx 21.9^\circ$.

9 Parametric contour generation

`scripts/generate_contour.py` is the end-to-end entry point:

```
PYTHONPATH=. python scripts/generate_contour.py \
    --rt 0.020 --epsilon 10 --length-pct 80 --gamma 1.4 \
    --theta-b-guess 21.87 --out builds/my_nozzle
```

It refuses to export unless the 2×10^{-3} gate passes (override with `--allow-unconverged`) and writes `contour.csv`, a binary `contour_3d.stl` surface of revolution, and the two figures below.

10 Verification and reliability

- **Oracle parity:** kernel march vs. NASA M3.5Perf grids (TT' 5×10^{-7} ; full 58-row grid; BD vs `LastKernel.out`).

Rao TOP contour $R_t=20.0$ mm $\epsilon=10$ $L=80\%$ $\gamma=1.4$
 $\max_scaled=1.16e-03$ $mass=-1.9e-09$ $len=-5.8e-10$ $kdf=0.30$

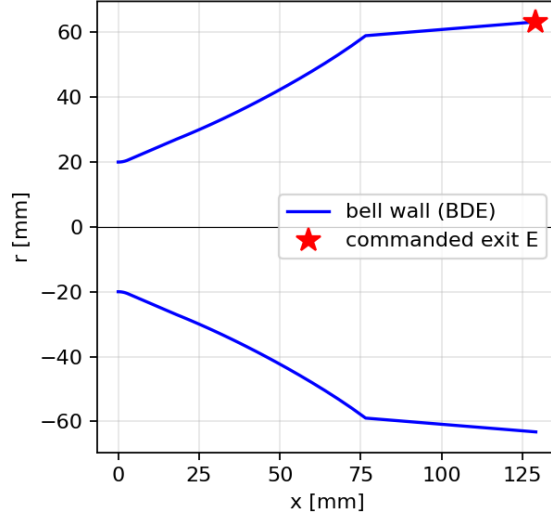


Figure 1: Converged Rao TOP profile for the reference design point ($R_t = 20$ mm, $\epsilon = 10$, 80%, $\gamma = 1.4$). The wall is the NASA BDE-region construction from the *solved* control surface; its final point lands exactly on the commanded exit (star).

Surface of revolution (exported to contour_3d.stl)

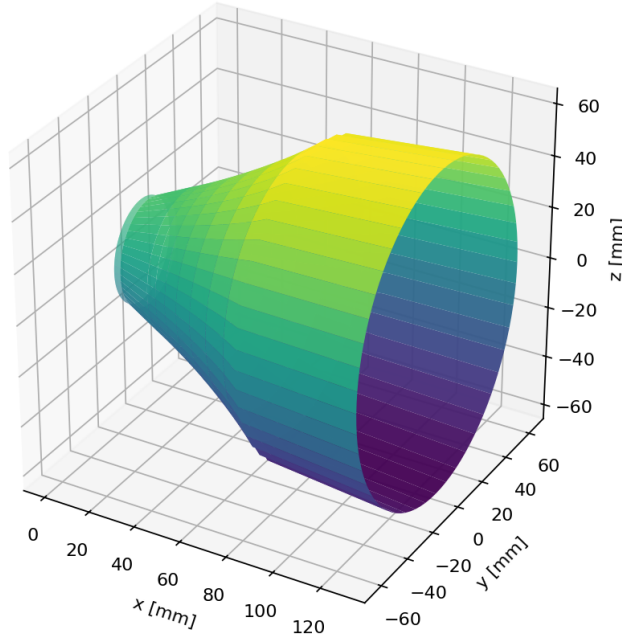


Figure 2: Surface of revolution of the same contour (exported as binary STL for CAD/printing).

- **Backend parity:** JAX vs NumPy residual $\sim 10^{-15}$ on real states; `jacfwd=jacrev=central` differences.

- **Gates:** J4 ($\max \text{scaled} \leq 2 \times 10^{-3}$) and the closed-wall test are ordinary (non-xfail) tests in `tests/test_jax_convergence.py`.
- **Reliability ladder:** GEOMETRIC_APPROXIMATION $\rightarrow$ MOC_COMPATIBLE $\rightarrow$ RAO_VARIATIONAL_RESIDUAL_SOLVED $\rightarrow$ BENCHMARK_VALIDATED; promotion past the first grade is currently held by the forward-MOC audit alignment (below).
- **Caveats:** every output carries `hardware_qualified=False`. No CFD, thermal, structural, manufacturing, or hot-fire validation is implied.

11 Remaining roadmap (formalised final steps)

1. **Default flip + re-baseline:** make `formulation="characteristic"`, `pin_d_theta=False`, the ladder, and `wall_method="bde"` the defaults; re-baseline the Phase-6/7 test expectations on the converged numbers.
2. **Forward-MOC audit alignment:** the verification pass still marches a spline wall from a $\text{chart-}\theta_N$ start line; align it with BDE-built walls so `moc_ok` can promote reliability honestly.
3. **J5 chart sweep:** run the converged configuration across the $(\varepsilon, \text{length}\%)$ grid against the published θ_N/θ_E tables (targets: RMS 1.5° , max 3°) — the BENCHMARK_VALIDATED gate.
4. **J6 gradient API:** `rao_sensitivities` — $\partial C_f / \partial(\text{params})$ and the per-node $|\partial C_f / \partial r_i|$ manufacturing-tolerance field via implicit differentiation through the converged LM solve (machinery already proven in `tests/test_jax.bvp.py`).
5. **Phase 12.4 march robustness:** extend the kernel past the $\theta_B \approx 24^\circ$ unit-process edge (tight throats), which also unlocks full state continuity at D (`pin_d_mach`).
6. **J3b:** `lax.scan` port of the march, making θ_B a solved unknown and enabling kernel-parameter sensitivities (start-line provenance studies).

References

- [1] G. V. R. Rao, “Exhaust Nozzle Contour for Optimum Thrust,” *Jet Propulsion* 28(6), 1958.
- [2] G. V. R. Rao, J. Beck, T. Booth, “Nozzle Optimization for Space-Based Vehicles,” AIAA 99-2584, 1999.
- [3] T. Rice, *2-D/Axisymmetric MOC Nozzle Design Code*, JHU/APL RTDC-TPS-481, 2003 (visible source: `MOC_GridCalc_BDE.cpp`).
- [4] J. R. Kliegel, J. N. Levine, “Transonic Flow in Small Throat Radius of Curvature Nozzles,” *AIAA J.* 7(7), 1969.
- [5] I. M. Hall, “Transonic Flow in Two-Dimensional and Axially-Symmetric Nozzles,” *QJMAN* 15(4), 1962.
- [6] J. D. Anderson, *Modern Compressible Flow*, 3rd ed., Ch. 11.
- [7] J. Östlund, *Flow Processes in Rocket Engine Nozzles with Focus on Flow Separation and Side-Loads*, KTH, 2002.

- [8] K. G. Guderley et al., “Continuous and discontinuous solutions for optimum thrust nozzles of given length,” *JOTA*, 1968.
- [9] J. Rader, T. Lyons, P. Kidger, “Optimistix,” arXiv:2402.09983.